DLIM Lecture 4: Detection Architectures

J. Chazalon

Session: Fall 2025

EPITA Research Laboratory (LRE)

Slides generated on November 24, 2025

Today's agenda

Computer Vision Tasks

Object Detection Techniques

Evolution of Region-Proposal-Based Detection Networks

Single-Stage Detection Networks

Anchor-Based Detection

Going Further

Lab Session: SSD Reimplementation

Remaining Work and Grading

Computer Vision Tasks

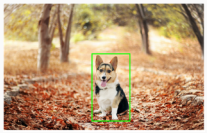
Classification

- Single label for the entire image.
- High-level understanding.
- No object locations.

$\rightarrow$ “DOG”

Localization

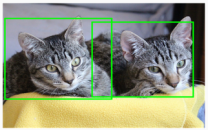
- Object position with bounding box.
- Single object.



→ “DOG” + bbox

Object Detection

- Identify and locate multiple objects.
- Includes classification and localization.



$\rightarrow$ bboxes (class, coords)

Semantic Segmentation

- Classify each pixel.
- Image divided by classes.

$\rightarrow$ classification map

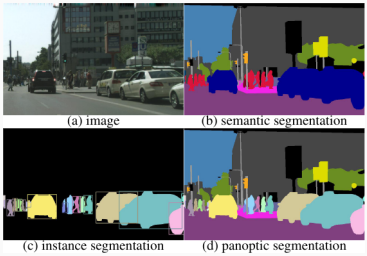
Instance Segmentation

- Unique label for each object instance.
- Pixel-level object masks.

$\rightarrow$ id map

Panoptic Segmentation

- Unifies semantic segmentation and instance segmentation.
- Assigns unique labels to all object instances and stuff classes.
- Provides pixel-level information for both objects and stuff.



$\rightarrow$ classification + id maps

Object Detection Techniques

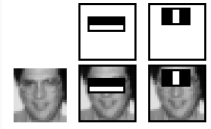
Pre-2010

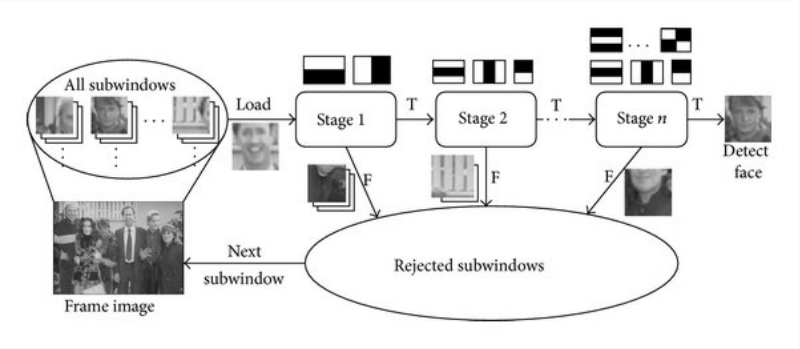
- **Haar cascades:** sequence of weak classifiers applied on the whole image
- **Sliding window approaches:** pre-compute features, then slide a classifier over the whole image, at multiple scales. Requires NMS*

*: NMS (Non-maximal suppression) is the removal of duplicate detections.

Haar cascades

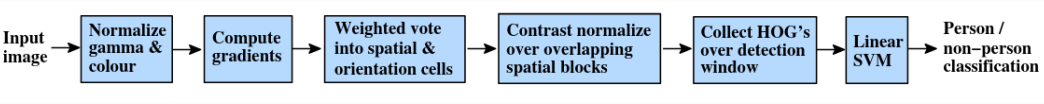
Paul Viola and Michael Jones, "Rapid object detection using a boosted cascade of simple features", CVPR'01





Sliding windows

Navneet Dalal and Bill Triggs, "Histograms of Oriented Gradients for Human Detection", CVPR'05



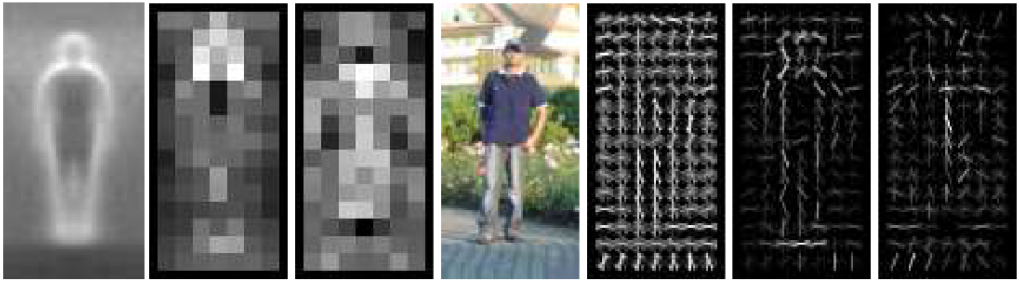


Figure 6. Our HOG detectors cue mainly on silhouette contours (especially the head, shoulders and feet). The most active blocks are centred on the image background just *outside* the contour. (a) The average gradient image over the training examples. (b) Each "pixel" shows the maximum positive SVM weight in the block centred on the pixel. (c) Likewise for the negative SVM weights. (d) A test image. (e) It's computed R-HOG descriptor. (f,g) The R-HOG descriptor weighted by respectively the positive and the negative SVM weights.

Region-Proposal-Based vs. Single-Stage Detection Networks

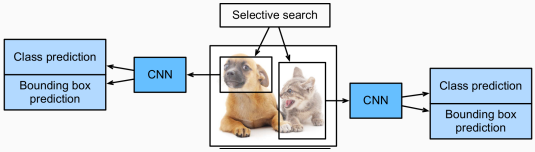
- **Region-Proposal-Based Detection Networks:**
  - Two-step process: Region proposal and classification.
  - Greater accuracy, especially in complex scenarios.
  - Flexibility in handling object sizes and shapes.
  - **Require ROI pooling**
  - Examples: R-CNN, Fast R-CNN, Faster R-CNN.
- **Single-Stage Detection Networks:**
  - One-step process for object detection.
  - Simplicity and speed.
  - Suited for real-time applications.
  - Examples: YOLO, SSD.

Both require alignment between ground truth and predicted regions.

Evolution of Region-Proposal-Based Detection Networks

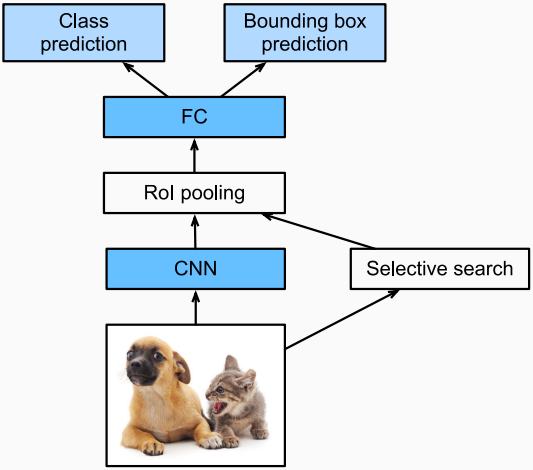
R-CNN (Region-based Convolutional Neural Network)

- **Key Contribution:**
  - Introduced region proposals for object detection.
- **Incremental Improvements:**
  - Used selective search to propose regions.
  - Each region was processed through a pre-trained CNN (for feature extraction) before classification.
  - Computationally expensive.



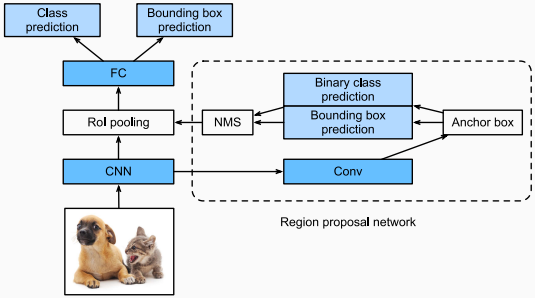
Fast R-CNN

- **Key Contribution:**
  - Unified the region proposal and feature extraction steps.
- **Incremental Improvements:**
  - Introduced RoI (Region of Interest) pooling layer.
  - Faster and more efficient.



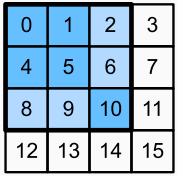
Faster R-CNN

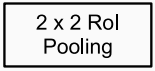
- **Key Contribution:**
  - Integrated region proposal generation into the neural network.
- **Incremental Improvements:**
  - Introduced the Region Proposal Network (RPN).
  - End-to-end trainable.
  - Achieved faster and more accurate detection.



RoI pooling

Views with autograd enabled!





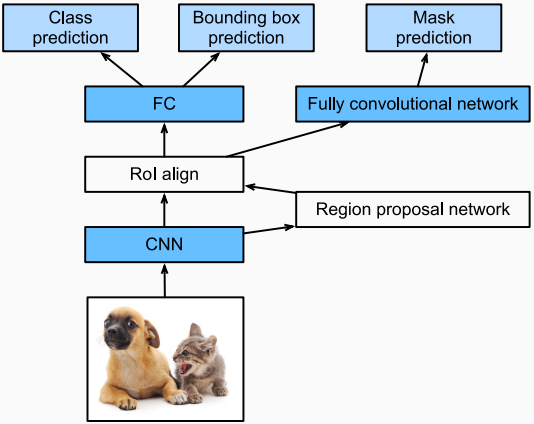

```
import torch
import torchvision

X = torch.arange(16.).reshape(1, 1, 4, 4)
rois = torch.Tensor([[0, 0, 0, 20, 20], [0, 0, 10, 30, 30]])
torchvision.ops.roi_pool(X, rois, output_size=(2, 2), spatial_scale=0.1)

>>> tensor([[[[ 5.,  6.],
              [ 9., 10.]]],
            [[[ 9., 11.],
              [13., 15.]]]])
```

Mask R-CNN

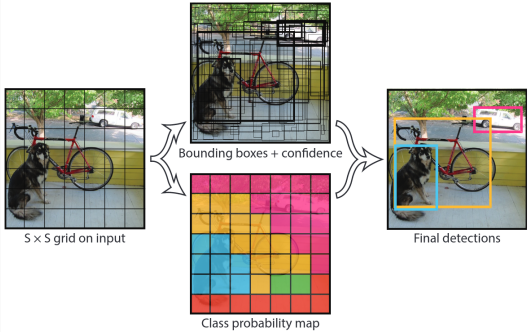
- **Key Contribution:**
  - Extended Faster R-CNN to include pixel-level instance segmentation.
- **Incremental Improvements:**
  - Added a mask prediction branch.
  - Enabled pixel-wise segmentation.
  - Simultaneous detection, localization, and segmentation.

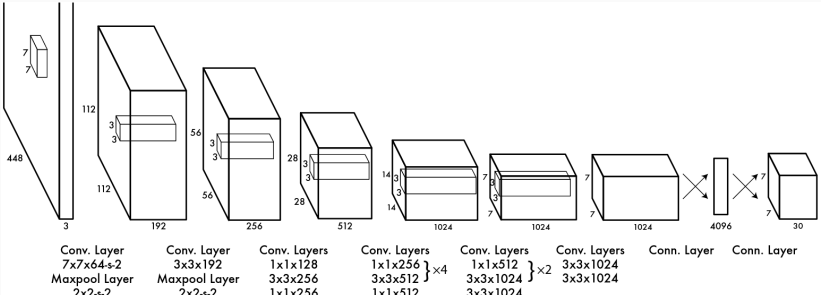


Single-Stage Detection Networks

YOLO (You Only Look Once)

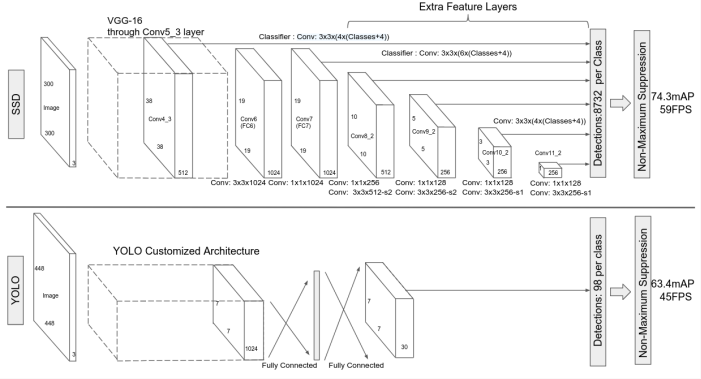
- **You Only Look Once (YOLO)** is a real-time object detection system that can detect multiple objects in an image in a single pass.
- **Principle:** divides an image into a grid and predicts bounding boxes, class probabilities, and objectness scores for each grid cell.
- **Known for:** speed and efficiency, suitable for real-time applications.
- **Note:** many versions





SSD (Single Shot MultiBox Detector)

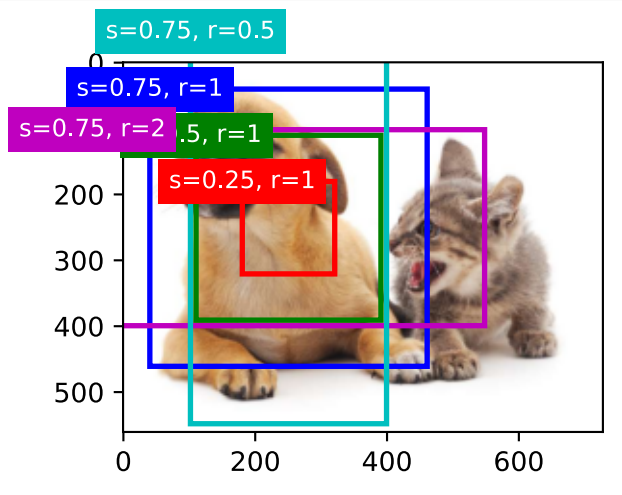
- **SSD (Single Shot MultiBox Detector)** is another popular single-stage object detection system.
- **Principle:** combines the benefits of multi-scale feature maps and default boxes (priors) to efficiently predict object locations and categories.
- **Known for:** capable of detecting objects of various sizes and aspect ratios in a single forward pass, providing a good trade-off between speed and accuracy.
- **Note:** widely used in real-time and embedded systems for object detection.



Anchor-Based Detection

What Are Anchors?

- Anchors are **predefined bounding boxes** of various sizes and aspect ratios.
- Used in anchor-based detection for predicting object locations and attributes.



Why Are Anchors Useful?

1. **Dense, Accurate Object Coverage:**
   - Ensures detection at various scales and positions in an image.
   - Accurate prediction of object locations.
2. **Efficient Computation:**
   - Reduces computational complexity by predicting anchor adjustments.
3. **Training Stability:**
   - Stable training with a consistent reference point + classification is easier than regression

Challenge with anchors

(and with regions predictions in general)

1. **Evaluation**
   - We must align the ground truth regions (bounding boxes, polygons, etc.) with the predicted ones.
   - This stage is **not differentiable**, and **does not need to be**.
   - Once the region-to-regions matching is established, losses can be computed and used to improve the parameters of the detection architecture.
2. **Choice of the base anchors**
   - Which shapes, sizes, orientations?
   - Depends on the domain, so either use simple configurations (like in the lab session) or perform a clustering over possible shapes on a representative sample.

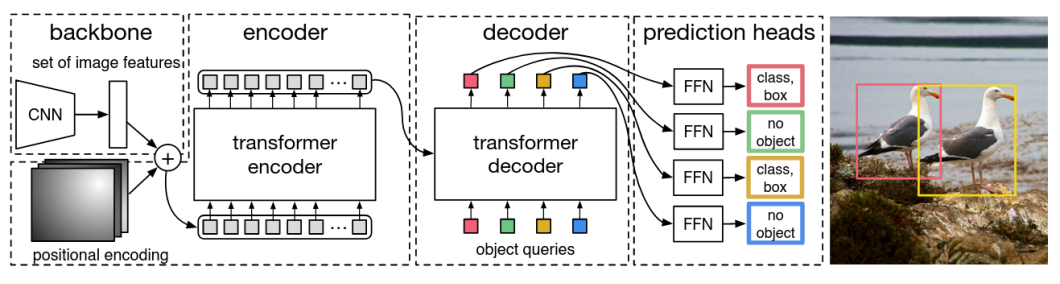
Going Further

Training Improvement

- Explore better loss functions for specific tasks, like Focal Loss.
- Consider techniques like Hard Online Example Mining (HOEM) to focus on challenging samples during training.

Transformer-Based Architectures

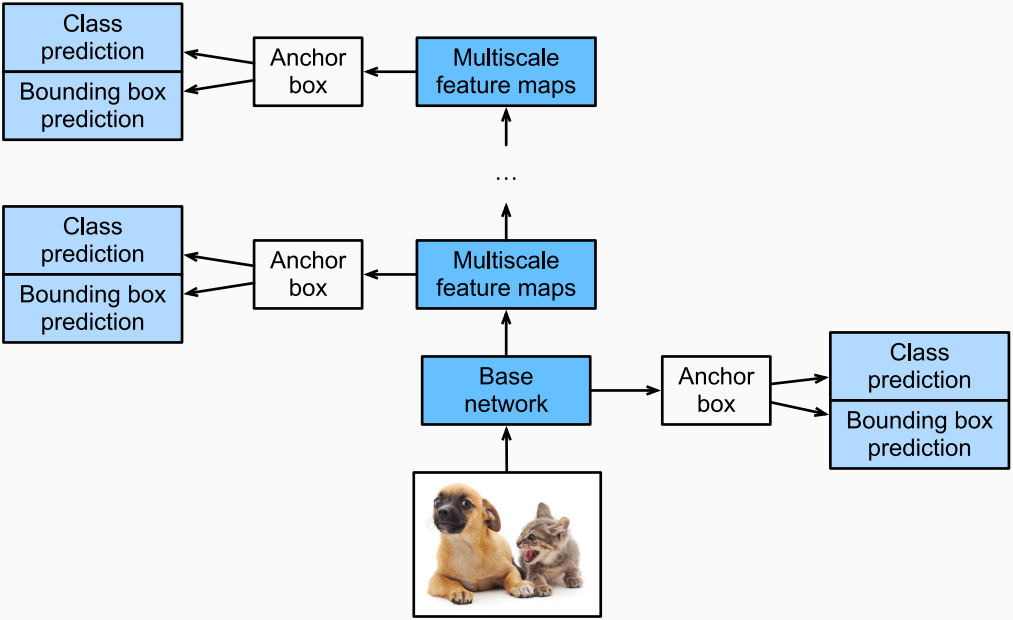
- **Segment Anything:** Segment Anything leverages vision transformers for versatile dense instance segmentation on a wide range of objects and scenarios.
- **DETR (Data-efficient Image Transformer):** DETR revolutionizes object detection, predicting both class and bounding box simultaneously for better data efficiency.
- *and much much more...*



Carion et al., "End-to-End Object Detection with Transformers", ECCV'20

Lab Session: SSD Reimplementation

Architecture

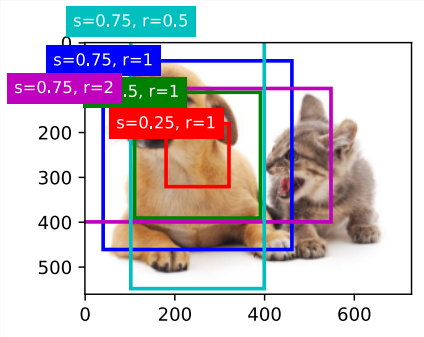


SSD is a multiscale object detection model

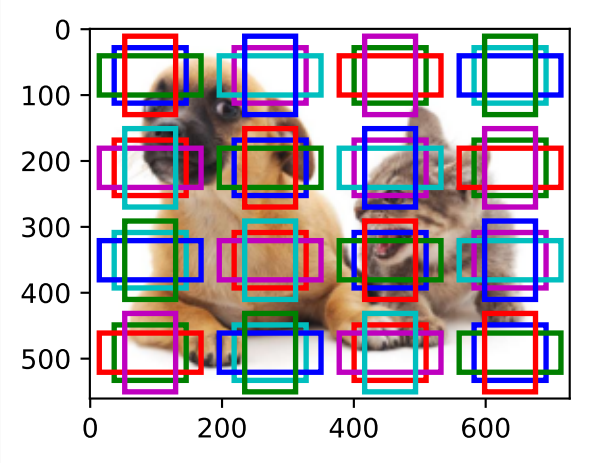
Anchors Generation

- `multibox_prior(data, sizes, ratios)` : box priors, $sizes + ratios - 1$
  Generate anchor boxes with different shapes centered **on each pixel**.
  Because generating all the combinations for each size and ratio would be too big,
  we only generate combinations containing either the first size or the first ratio:

$$(s_1, r_1), (s_1, r_2), \ldots, (s_1, r_m), (s_2, r_1), (s_3, r_1), \ldots, (s_n, r_1).$$

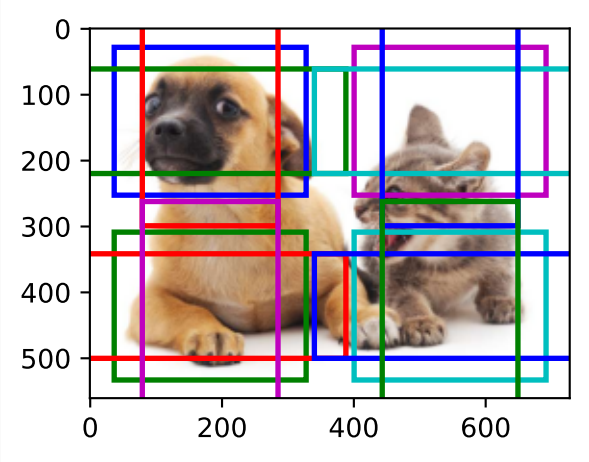


Multi-scale Features Maps



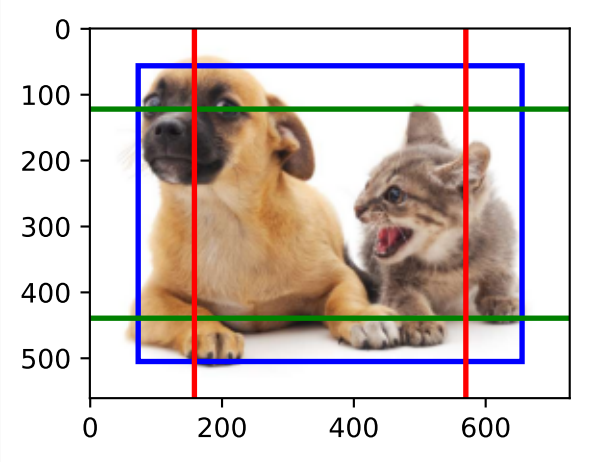
Scale 1:1

(Only a selection of anchor boxes is displayed!)



Scale 1:2

(Only a selection of anchor boxes is displayed!)



Scale 1:4

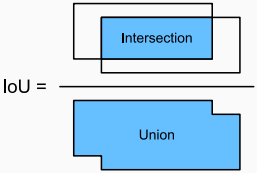
(Only a selection of anchor boxes is displayed!)

Anchors Matching with Ground-truth

- IoU computation: `box_iou(boxes1, boxes2)`

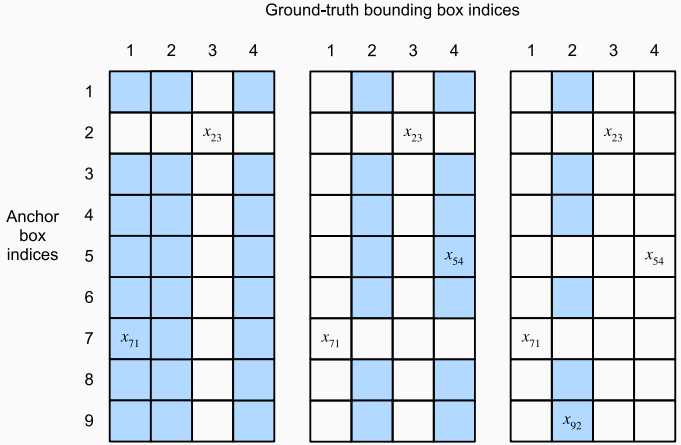
$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Measures the matching between an anchor and a box (or between two boxes)



- matching algorithm (tricky): `assign_anchor_to_bbox(ground_truth, anchors, device, iou_threshold=0.5)` Assign closest ground-truth bounding boxes to anchor boxes

    1. Iteratively assign target boxes to anchors by decreasing IoU, forbid matching the same anchor or target box again. *Illustrated below* ↓
    2. When all target boxes are assigned, assign remaining anchor boxes as extra matches only if they reach a minimum overlap.



Generate Actual Targets

- `offset_boxes(anchors, assigned_bb, eps=1e-6)`: $(\Delta_x, \Delta_y, \log(\Delta_w), \log(\Delta_h))$
  Computes actual anchors offsets given the assigned target box.
- `multibox_target(anchors, labels) -> bbox_offset, bbox_mask, class_labels`:
  Main function which generates targets for each anchor using ground-truth bounding boxes

Predicting Bounding Boxes with Non-Maximum Suppression

At test time only, not needed during training.

- `nms(boxes, scores, iou_threshold) -> keep`: filter overlapping boxes:
  Select box with highest confidence, keep it, find overlapping boxes, remove them, repeat.
- `multibox_detection(cls_probs, offset_preds, anchors, nms_threshold=0.5) -> boxes`: final detection, given:
  - `cls_probs` and `offset_preds`: network predictions
  - `anchors`: anchor definitions for this network
  - `nms_threshold`: minimum overlap between boxes to merge them

Losses

- for classes: cross-entropy (pushes toward 1 or 0)
- for bounding boxes: L1 Loss — could be MSE too (minimizes distance to expected value)

Remaining Work and Grading

Remaining Work and Grading

TODOS:

- lab session
  - contribute the collective dataset ← **submission on Moodle, graded**
  - understand SSD in depth
  - rewrite (naively) parts of the code
  - train on the new dataset (add: validation measure + early stopper + model saver + seeding)
  - process the test set
  - submit your predictions on the test set ← **submission on Moodle, graded**
- quiz 4 ← **submission on Moodle, graded**

Deadlines:

- **Wednesday 19:00** for the dataset
  → 20 images p. person, with annotations, only 1 logo p. img
- Download the complete, new dataset by **next Thursday** (will send a message on Moodle)
- **Submit predictions for test set by Tue., Dec 2nd.** I'll (try to) grade everything on Wed., Dec 3rd.

Do not wait for the full dataset to be ready to validate your approach!